PRAMANA

CodeStar 30 · Round 2 Build and Verification Report

Team 20ananya.pandey | 29 October 2026 | Evidence-led brand governance demo

DEPLOYMENT SNAPSHOT · Frontend is live at 20ananyapandey.vercel.app. The page loaded and reported API connected; the backend /health endpoint returned status ok. The public analyze-to-history journey has not been independently exercised end to end.

Executive summary

Pramana is a focused pre-publication review demo for fictional LumaDesk. It checks draft copy against four code-defined rules and returns evidence a reviewer can inspect before choosing to approve, request revision, or reject. The system does not publish content or use an LLM in its analysis path.

The implemented value is traceability: a finding must quote the evaluated draft and point to an active seeded rule and source excerpt. This makes the warning inspectable; it does not prove every cited claim is true or replace a human reviewer. Source: finding validator and review service.

Project and live links

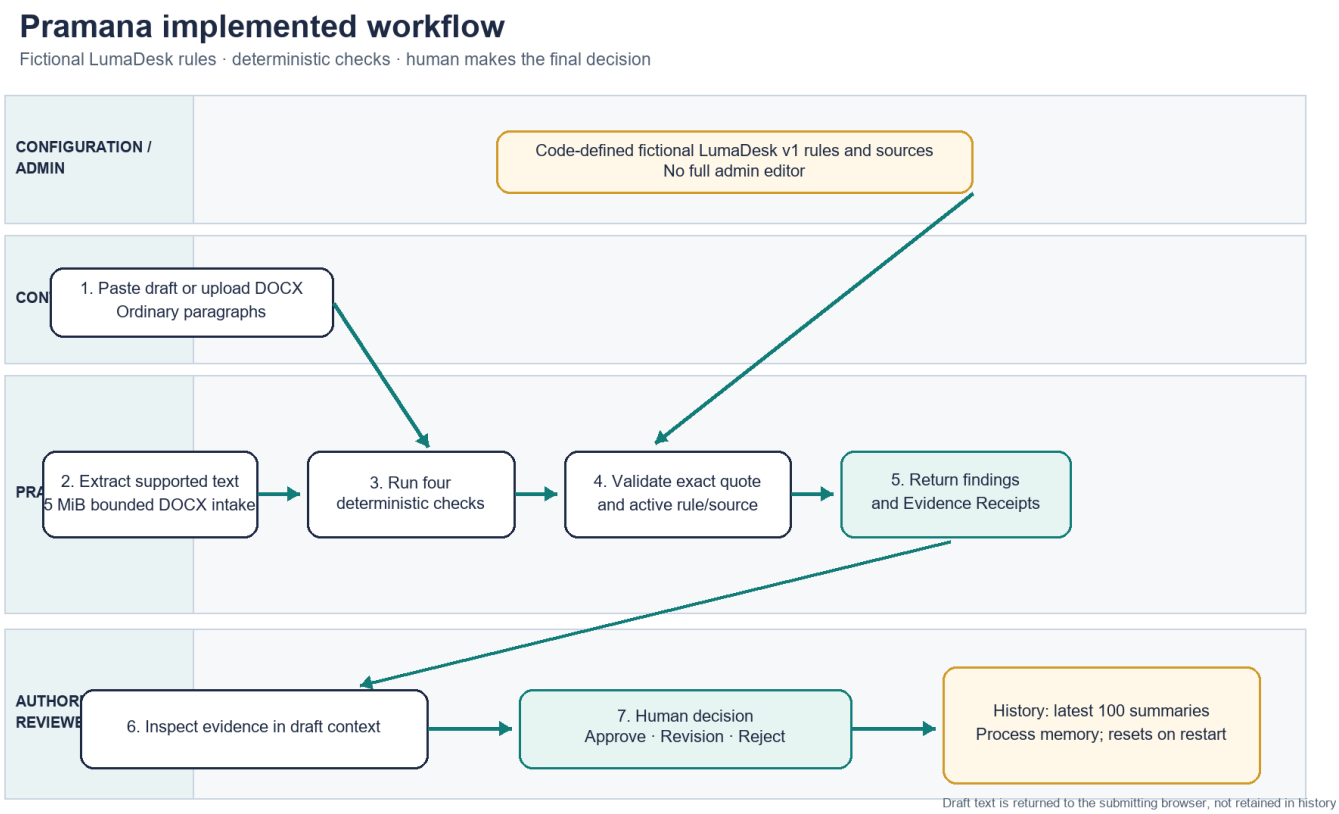
Resource	Verified link / status
Public source repository	github.com/ananyap-20/Pramana
Live frontend	20ananyapandey.vercel.app
Backend health	pramana-backend.vercel.app/health
Public workflow test	Frontend/API connectivity observed; public analyze → decision → history journey not independently tested.
Project category	Evidence-led, human-in-the-loop brand governance

Submission scope

This report describes the shipped implementation and current verification evidence. Local automated checks and a local browser workflow pass; the live page and API are reachable, but full hosted workflow verification remains open. The report does not claim that a form was submitted. The official event page lists the Round 2 window as 26–28 October 2026; the report date is after that listed window. Sources: CodeStar 30 official event page and repository test matrix.

Implemented workflow

The diagram below follows the actual code path and separates code-defined configuration, creator input, system checks, and the authorized reviewer. The reviewer, not the system, makes the final decision.



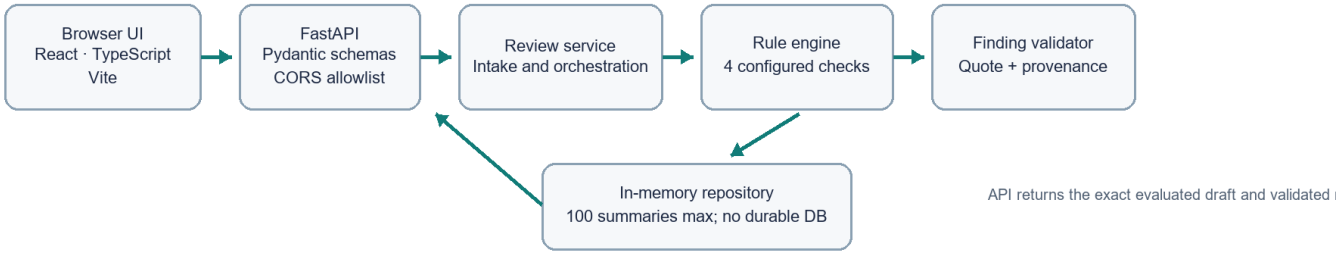
The four checks cover discouraged terminology, configured prohibited phrases or claims, one configured plan-fact conflict, and a required disclaimer. Seeded LumaDesk material is fictional. DOCX extraction supports ordinary paragraphs only; uploaded content is treated as untrusted input and bounded by size and extraction limits.

Implementation sources: seed rules, rule engine, and DOCX intake.

Architecture and shipped features

Architecture

Pramana technical architecture



The browser uses React, TypeScript, and Vite. FastAPI validates requests with Pydantic schemas and calls a review service, deterministic rule engine, evidence validator, and in-memory repository. The API returns the exact evaluated draft and validated receipts to the browser; the repository stores summaries and decision events, not draft text. `DATABASE_URL` is reserved but unused. Source: API, repository, and frontend API client.

Feature completion

Feature	Status	Evidence and boundary
Paste text and analyze	Implemented	Calls FastAPI and shows results for the current run.
DOCX paragraph intake	Implemented	Ordinary paragraphs only; bounded upload size.
Four deterministic rule checks	Implemented	Seeded fictional LumaDesk v1 configuration.
Evidence Receipts and quote/provenance validation	Implemented	Mismatched quotes or inactive rule/source context are rejected.
Human review decisions	Implemented	Approved, revision, or rejected; decision history is API-backed.
Shared history	Partial	At most 100 summaries in process memory; resets on restart.
Admin rule editor, auth, durable audit, analytics	Not implemented	Roadmap work, not shipped capabilities.

Evidence Receipts contain the flagged passage when applicable, rule and source identifiers/versions, source excerpt, explanation, evidence status, and suggested action. The validator ensures the quote and provenance match the evaluated draft and active seeded configuration; it cannot establish the truth of a source or a claim by itself.

Verification, changes, and boundaries

Checks run against the inspected repository

Check	Result	What it shows
Backend regression suite	54 passed; 0 failed; 1 deprecation warning	Synthetic configured cases only, not real-world accuracy.
Python compilation	Passed	Backend modules compile.
Vite production build	Passed (Vite 8.3.5)	TypeScript and Vite build succeeded.
Oxlint	Completed; 2 warnings	Both are React set-state-in-effect warnings in history and rules pages.
Local browser/API journey	Passed in test matrix	Analyze → receipts → decision → history; local API calls returned HTTP 200.
Live deployment check	Frontend loaded; API connected; /health status ok	Hosted analyze-to-history workflow not independently run.
Responsive inspection	Partial	731 CSS px inspected; 390 px not separately checked.

The 54/54 result is only the pass rate for this synthetic regression suite. It is not a precision, recall, detection-accuracy, latency, or productivity metric. The single warning comes from the Starlette/httpx TestClient integration.

Changes from the supplied Round 1 plan

The evidence-first core was retained. The build turned simulated sign-off and sample history into API-backed reviewer decisions and run summaries, added bounded DOCX intake, receipt context, and tests. PostgreSQL, durable audit records, full authentication, advanced file intake, visual brand checks, and analytics remain deferred. The archive did not include an independently verifiable Round 1 baseline, so no measured progress comparison is claimed. Sources: implementation notes and roadmap.

Security, privacy, AI, and reuse

The public demo has no authentication, authorization, organization isolation, or rate limiting. Visitors share history summaries; anyone who knows a run ID may change its decision. Draft text is processed by the API and returned to its submitting browser, but is not retained in history. Do not submit confidential copy. History is volatile and is not an immutable audit log. The app provides neither general-purpose fact checking nor legal advice, and it does not automatically publish or rewrite content.

OpenAI Codex assisted with development review and this report. The live analysis path uses no LLM or external analysis API. The repository has no project-level license; third-party dependencies remain subject to their own license terms. No copied-code or originality claim is made beyond the project's disclosed source history.

Round 3 commitments and submission readiness

These are proposed improvements, not shipped features. They match the project roadmap and are substantial, measurable work.

Commitment	User value and implementation	Proposed acceptance measure and risks
1. Richer document and image intake	Add bounded PDF/image extraction with page or region provenance, confidence indicators, and human confirmation for low-confidence OCR. Feed only confirmed text into the existing validator.	On a labeled fictional set, extract at least 95% of expected paragraphs for supported text PDFs; low-confidence OCR is visibly marked. Risks: OCR quality, licensing, resource use, malicious or oversized files.
2. Visual brand and accessibility checks	Add measurable checks for approved logos/colors, rendered text contrast, and missing alt text; keep subjective design judgments with reviewers.	Every finding includes reproducible element/value measurements; zero unexplained misses on an exact fixture suite. Dependencies: approved assets, palette, contrast standard, rendering engine.
3. Persistent audit and analytics	Define retention and access policy; add authentication, organization isolation, relational storage, append-only decisions, and basic analytics without retaining draft text by default.	Decisions survive restart; cross-organization access tests pass; aggregates reconcile to source events. Risks: identity, backups, migrations, privacy and operational cost.

Local setup

Requirements: Python 3.12+, uv, and Node.js compatible with the checked-in Vite version. At the repository root run `uv sync --locked --all-groups`; start the API with `uv run uvicorn app.main:app --app-dir backend --env-file .env --host 127.0.0.1 --port 8000`. In another terminal, from `frontend`, run `npm ci`, set `VITE_API_BASE_URL=http://127.0.0.1:8000`, then `npm run dev`. See the README for configuration and safe-use notes: README.

Submission links and references

Public repository · README · test matrix

workflow diagram · Round 3 roadmap · evaluation set

Official CodeStar 30 event page · Round 2 submission form supplied in report prompt

Final status

Pramana is available on the public Vercel frontend and its API health endpoint is reachable. Local functionality is backed by a passing test suite and a previously recorded local browser journey. The hosted interface reports API connectivity, but the full hosted review workflow should still be exercised before describing that journey as verified. Keep all sample content fictional. The event schedule published by We Code Coders lists Round 2 as 26–28 October 2026 and says judges need an openable, usable project link from Round 2; this report does not state that a submission has been made.

Submission guidance note: the Round 2 form text provided for this report says one team-leader submission, while the public event page currently says each participant submits individually. Follow the live round-specific form instructions and resolve the discrepancy with the organiser if both directions appear for your entry.